Part 1 assignments

Course: Agentic Software Engineering, Part 1: The Model as a Measured Component.

There are **two assignments**. Both use a small local model run by Ollama, so nothing costs real money. Dates and the submission method are set by your instructor.

#	Title	One-line summary
1	SQL calculation agent	An agent that answers calculation questions by calling tools against a SQL database
2	English vs Bangla token cost	50 English and 50 Bangla prompts with the same meaning, run through the model and compared on tokens and cost

Shared setup

- Model: `qwen2.5:3b` (Qwen 2.5, 3 billion parameters) served by Ollama. This is the minimum size. A larger model is allowed, but say which one you used.
- Language: Python is recommended. Use the standard library plus `sqlite3` where you can.
- Reuse the Class 1 wrapper (`agentlab/lm.py`) if you want. Its ledger already records tokens per call.
- Test first. Write a test, watch it fail, then write the code.

Assignment 1: Build an AI agent with local models

Goal

Build an agent that answers **calculation questions** by calling tools, not by guessing the numbers itself. The numbers come from a SQL database that you create with dummy data.

Requirements

1. **Model.** Run `qwen2.5:3b` (or larger) through Ollama. No hosted APIs.
2. **At least two tools:**
    - **SQL generation tool.** Given a question and the database schema, it produces a SQL query. Design choice: this can be a tool that makes its own model call, or the schema can be passed to the agent and the agent writes the SQL as the tool argument. Explain which you chose and why.
    - **SQL executor tool.** Runs a SQL query against the database and returns the rows (or a clear error message).
    - **More tools are welcome**, for example a calculator, a date helper, a schema lookup, or a table lister.
3. **Calculation by tools.** Questions such as totals, averages, counts, differences, percentages and growth must be answered by calling tools. The final answer must be based on what the tools returned.
4. **Database.** Create a SQL database (SQLite is fine) with dummy data:

- at least **3 related tables** (for example `customers`, `products`, `orders`, `order_items`);
- at least **100 rows in the biggest table**;
- a script that creates and fills it from a fixed random seed, so anyone can rebuild the same data.

5. **Agent loop.** Write the loop yourself (no agent SDK or framework). It must have:
   - a **step cap** (stop after N steps);
   - a **budget or token cap**;
   - a clear **final status** for each run (for example `ok`, `step_cap`, `error`).
6. **Safety.** The executor must be **read-only**: it accepts only `SELECT`, rejects everything else (`DROP`, `DELETE`, `UPDATE`, `INSERT`, and several statements in one string), and applies a row limit. Open the database in read-only mode if your library allows it.
7. **Error handling.** If the SQL is wrong, return the error text to the model so it can try again, within the step cap.

Test set

Write **at least 15 questions** with known correct answers, computed by your own SQL, not by the agent. Cover a range:

- simple (one table, one number: "How many customers are there?");
- joins ("Total revenue for customer 12?");
- grouping ("Which product category earned the most?");
- calculation ("Average order value, and what percent above the minimum?");
- at least 2 questions that need **more than one tool call**;
- at least 2 questions the database **cannot answer** (the agent should say so instead of making up a number).

Run each question **3 times**.

What to submit

1. Source code, including the database builder and your tests.
2. The database file or the script that rebuilds it.
3. The question set with expected answers.
4. The run log: one record per run (question, tool calls, SQL written, final answer, status, tokens, time).
5. A short report (1 to 2 pages) with:
   - the architecture in a diagram or a few lines;
   - **pass@1** and **pass^3** over your questions;
   - the three most common failure types, with an example of each;
   - one change you made to improve it and the before and after numbers.
6. A README that tells us how to run everything in under 5 commands.

Marking guide (100 points)

Area	Points	What we look for
Working agent with 2 or more tools	25	Tools are called by the model, the loop is yours, the answer uses tool results
Database and read-only safety	15	3 or more tables, seeded dummy data, non-SELECT statements are rejected (with tests)
Loop controls	10	Step cap, budget cap, clear final status
Evaluation	25	15 or more questions, 3 trials each, pass@1 and pass^3 reported, expected answers computed independently
Failure analysis	10	Failures grouped and explained honestly
Tests and code quality	10	Tests written first, runnable with one command
README and report	5	Someone else can reproduce your numbers

Bonus (up to 5): an extra tool, or a comparison of `qwen2.5:3b` with a larger model on the same questions.

Hints

- Ollama's chat endpoint accepts a `tools` list. Not every model supports it, so test yours with a tiny tool first. If you prefer, ask the model for JSON (`{"tool": ..., "args": ...}`) and parse it yourself, as in Class 3.
- Put the table definitions in the prompt. A small model writes much better SQL when it can see the real column names.
- Small models make mistakes: invented column names, missing joins, wrong arithmetic. That is why the tool, not the model, does the calculation.
- Do not trust one run. Run three times and report the spread.

Assignment 2: English vs Bangla token cost

Goal

Measure how much more (or less) a Bangla prompt costs than an English prompt with the **same meaning**, using a local model run by Ollama.

Requirements

1. **Dataset.**
    - **50 English prompts** and **50 Bangla prompts**, where prompt i in Bangla has the **same meaning** as prompt i in English (50 aligned pairs).
    - Mix the kinds of prompts: questions, instructions, short explanations, simple arithmetic, summarising a given sentence, and a few longer prompts (3 to 5 sentences). Do not make all 50 the same length.

- Save as one file (CSV or TSV) with columns: `id`, `category`, `english`, `bangla`.
- Write the Bangla yourself or check it carefully. If you use a translation tool, review every pair and say so in your report.

2. **Token counting system.** Build a small, reusable function or module that returns, for each call, the number of **input (prompt) tokens** and **output (completion) tokens** as Ollama counts them. Ollama returns these in its response as `prompt_eval_count` and `eval_count`. State clearly how you handle:
   - the extra tokens added by the chat template or system prompt (measure it once with a tiny prompt and report the baseline, or use the same fixed system prompt for every call);
   - repeated prompts: Ollama can reuse a cached prompt, which can make `prompt_eval_count` smaller on a second run, so run each prompt in a fresh state or check for this and explain what you did.
3. **Runner.** A program that sends all 50 English and all 50 Bangla prompts to the model, with the **same settings** for both languages (model, temperature 0 or a fixed value, seed, `max_tokens`), and records every result. Use the same system prompt for both. Do not tell the model to "answer in the same language as the question" for one language and not the other, unless you say so and justify it.
4. **Cost.** Compute the cost of each call with the course's notional prices (`$0.15` per million input tokens, `$0.60` per million output tokens), and total the cost for each language.
5. **Comparison.** For each pair and overall, compare Bangla with English.

Measures to report

Measure	How
Input tokens, English and Bangla	per pair, plus total
Output tokens, English and Bangla	per pair, plus total
Input cost ratio	Bangla input tokens ÷ English input tokens for the same meaning
Total cost ratio	Bangla total cost ÷ English total cost
Spread	mean, median, minimum, maximum and a 95% interval of the per-pair ratio (a bootstrap interval is fine)
Latency	seconds per call, English vs Bangla
Quality check	for 10 pairs of your choice, say whether the Bangla answer is as correct and useful as the English one

Also report: which categories show the largest gap, and one prompt where the ratio surprised you, with your explanation.

What to submit

1. `dataset.tsv` (or `.csv`): the 50 aligned pairs.
2. The token-counting module and the runner program, with at least a few tests (use a fake backend to test the counting and cost code).
3. `results.csv` (or `.jsonl`) with one row per call: `id`, `language`, `prompt`, `output`, `input_tokens`, `output_tokens`, `cost_usd`, `latency_s`.
4. `comparison.csv`: one row per pair with English and Bangla tokens and the ratios.
5. A short report (1 to 2 pages) with a table of the measures above, one chart (ratio per pair or per category), and your conclusions.
6. A README to reproduce the results.

Marking guide (100 points)

Area	Points	What we look for
Dataset	20	50 + 50 aligned pairs, same meaning, varied categories and lengths
Token-counting system	20	Uses Ollama's counts, handles template overhead and prompt caching, tested
Runner and recorded outputs	20	Same settings for both languages, every input and output saved
Cost and comparison	20	Correct arithmetic, ratios per pair and overall, spread reported (not only an average)
Analysis and report	15	Clear table and chart, honest conclusions, one surprising case explained
Reproducibility	5	Others can rerun it and get similar numbers

Bonus (up to 5): repeat the experiment on a second model and compare tokenizers.

Hints

- Bangla uses 3 bytes per character in UTF-8, so a tokenizer that has seen little Bangla splits the text into many small pieces. Your numbers will show how much, for this model.
- Do not quote ratios from lectures as facts. Report **your measured** ratio and its spread.
- Make sure each Bangla prompt really means the same as its English partner. A longer or more detailed Bangla version breaks the comparison.
- The answer length is not under your control unless you set `max_tokens`. Report output tokens separately from input tokens, and say if `max_tokens` cut some answers short.

Rules for both assignments

- Your own work. If you use a coding assistant, you write (or approve) the test first and keep a three-line log: what it wrote, which test proves it, what you reviewed.
- Report spread, not one run.
- Say plainly what did not work. A clear failure analysis earns marks; hiding failures loses them.
- Keep prices **notional**: they are made-up dollars for a free local model, used so we can practise the arithmetic.
- Do not commit the model files. Say which Ollama model tag you used.

Submission checklist

- [ ] Code runs with the commands in the README
- [ ] Tests pass
- [ ] Data files included (database or builder, dataset)
- [ ] Run logs and results files included
- [ ] Report with numbers and the model tag
- [ ] Short log of any AI-assistant use